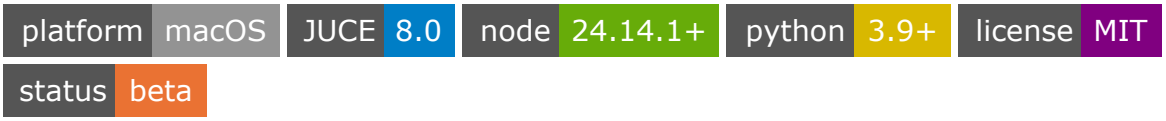
MOTIFF

AI-powered audio generation plugin for Ableton Live



MOTIFF is an open-source VST3/AU plugin that lets you send audio directly to a local AI model and receive generated audio back — all from inside your DAW. Built with JUCE, it integrates with Ableton Live via the Extensions SDK to automatically place generated audio onto a new track.

What it does

- Load a WAV, AIFF, or FLAC file into the plugin
- Write a text prompt describing the transformation
- Choose a generation mode:
  - **Append** — select a region of your audio for the model to use as reference, then generate new audio that continues or responds to it
  - **Restyle** — transform the audio with a creativity dial
- Hit **Generate** — the plugin sends your audio and parameters to a local AI server specially trained to a Maqam dataset
- The result appears as a **new audio track in Ableton Live automatically**

Architecture

```
JUCE Plugin (VST3/Standalone)
     │
     ├── Sends audio + parameters to local Python server
(HTTP POST)
     │
     └── Writes trigger file to disk
               │
               └── Ableton Extension (TypeScript)
                         │
                         └── Detects trigger → creates new Audio
Track in Live
```

The plugin, server, and extension are three separate components that communicate via localhost HTTP and the filesystem.

Requirements

All components

- macOS (Apple Silicon or Intel)
- Ableton Live 12 Beta (for the auto-track feature)

JUCE Plugin

- JUCE 8
- Xcode 14+

Python Server

- Python 3.9+
- Flask (`pip3 install flask`)
- Your trained Stable Audio model

Ableton Extension

- Node.js 24.14.1+
- Ableton Live 12 Beta

Installation

1. Clone the repo

```
git clone https://github.com/yourname/motiff.git
cd motiff
```

2. Build the JUCE plugin

1. Download JUCE and note where you put it
2. Open `AINewPlug/AINewPlug.jucer` in Projucer
3. Go to **Projucer → Global Paths** and set the path to your JUCE folder
4. Make sure **VST3** and **Standalone** are ticked under Plugin Formats
5. Click **Save and Open in IDE**
6. In Xcode, hit **Cmd+B** to build

Copy and sign the VST3:

```
cp -r AINewPlug/Builds/MacOSX/build/Debug/AINewPlug.vst3 ~/Library/Audio/Plug-Ins/VST3/
sudo codesign --force --deep --sign - ~/Library/Audio/Plug-Ins/VST3/AINewPlug.vst3
```

Note: Build in **Debug** mode for development. The VST3 built in Release mode may not load in Ableton without a paid Apple Developer certificate.

3. Set up the Python server

```
cd server
pip3 install flask
```

The server expects a POST request at `http://127.0.0.1:8080/process` with:

Field	Type	Description
`audio`	file	Input WAV file
`mode`	string	`"append"` or `"restyle"`
`prompt`	string	Text description
`start`	float	Selection start in seconds
`end`	float	Selection end in seconds
`creativity`	float	0.0–1.0 (restyle mode only)

It should return a WAV file as the response body.

To run with the echo test server (no model required):

```
python3 server/server.py
```

To use your own model, replace the processing logic in `server/server.py`.

4. Set up the Ableton Extension

```
cd AbletonExtension
npm install
```

Create a `.env` file (or update the existing one):

```
EXTENSION_HOST_PATH=/Applications/Ableton Live 12 Beta.app/
Contents/Helpers/ExtensionHost/ExtensionHostNodeModule.node
```

Running

Start everything in this order:

```
# Terminal 1 — Python server
python3 server/server.py

# Terminal 2 — Ableton extension (after opening Ableton
Live 12 Beta)
cd AbletonExtension
npm start
```

Then in Ableton Live 12 Beta:

1. Load **AINewPlug** VST3 on an audio track
2. Load an audio file in the plugin UI
3. Write a prompt, set your selection, hit **Generate**
4. A new **AI Generated** track will appear automatically with your result

Project structure

```
motiff/
├── AINewPlug/                          # JUCE plugin (VST3 +
Standalone)
│   └── Source/
│       ├── PluginProcessor.h/.cpp      # Audio engine +
server communication
│       └── PluginEditor.h/.cpp         # UI — waveform,
controls, transport
│
├── AbletonExtension/                   # TypeScript
extension for Ableton Live
│   └── src/
│       └── extension.ts               # Watches for
generated audio, creates track
│
└── server/
    └── server.py                       # Local inference
server (Flask)
```

How the Ableton integration works

When generation completes, the plugin writes two files to the system temp directory:

- `AINewPlug/ai_output.wav` — the generated audio
- `AINewPlug/ai_output_ready.txt` — a trigger file

The Ableton Extension polls for the trigger file every second. When it detects it, it uses the Ableton Extensions SDK to:

1. Create a new Audio Track named "**AI Generated**"
2. Place the generated WAV as a clip at position 0
3. Name the clip "**AI Output**"

Known limitations

- **macOS only** — Windows and Linux are not currently supported. The JUCE plugin and Ableton Extension SDK are macOS-specific in this implementation.
- **Ableton Live 12 Beta required** for the auto-track feature. The plugin works as a standalone app or in other DAWs (Reaper, Logic etc) but will not auto-create tracks without the Beta's Extension Host.
- **Model not included** — the Python server is a template. You need to supply your own trained audio model. The repo ships with an echo server for testing the pipeline end-to-end.
- **No real-time processing** — generation is triggered manually via the Generate button. Live audio input is not currently supported.
- **Ad-hoc code signing only** — the VST3 is signed with a self-signed certificate. Some DAWs (notably Ableton 12 production) may block it. Use Reaper or Ableton Beta for development.
- **Temp file path** — the generated audio is written to the macOS temp directory (`$TMPDIR/AINewPlug/`). This path varies per machine and per session, which means the Ableton Extension must run on the same machine as the plugin.
- **Single instance** — running multiple instances of the plugin simultaneously is untested and may cause temp file conflicts.

Contributing

Pull requests welcome. Please open an issue first to discuss any major changes.

Acknowledgements

- JUCE — the C++ framework that makes cross-platform audio plugin development possible
- Ableton Extensions SDK — early access beta SDK that enabled the auto-track integration
- Flask — the lightweight Python server powering the inference endpoint
- Stable Audio — the audio generation model architecture this plugin was built around
- Cretaed by: Chris Powers, Thiago Santos, Jad Al-Masri, Rithik Kundu, Michael Tomasi
- Model Training Assistance: CJ Carr
- Claude (Anthropic) — AI pair programmer used throughout development

License